Task 1: Wazuh Installation and File Integrity Monitoring (FIM)

SUBMITTED BY: RAJA MUHAMMAD BILAL ARSHAD

ITSOLERA TEAM:EPSILON

SUBMISSION DATE:12TH JANUARY 2026

1. Objective.....2
2. Implementation Strategy.....2
3. Tuning & Reducing False Positives.....6
4. Testing & Validation.....6
5. Challenges Faced.....7
6. Conclusion.....7

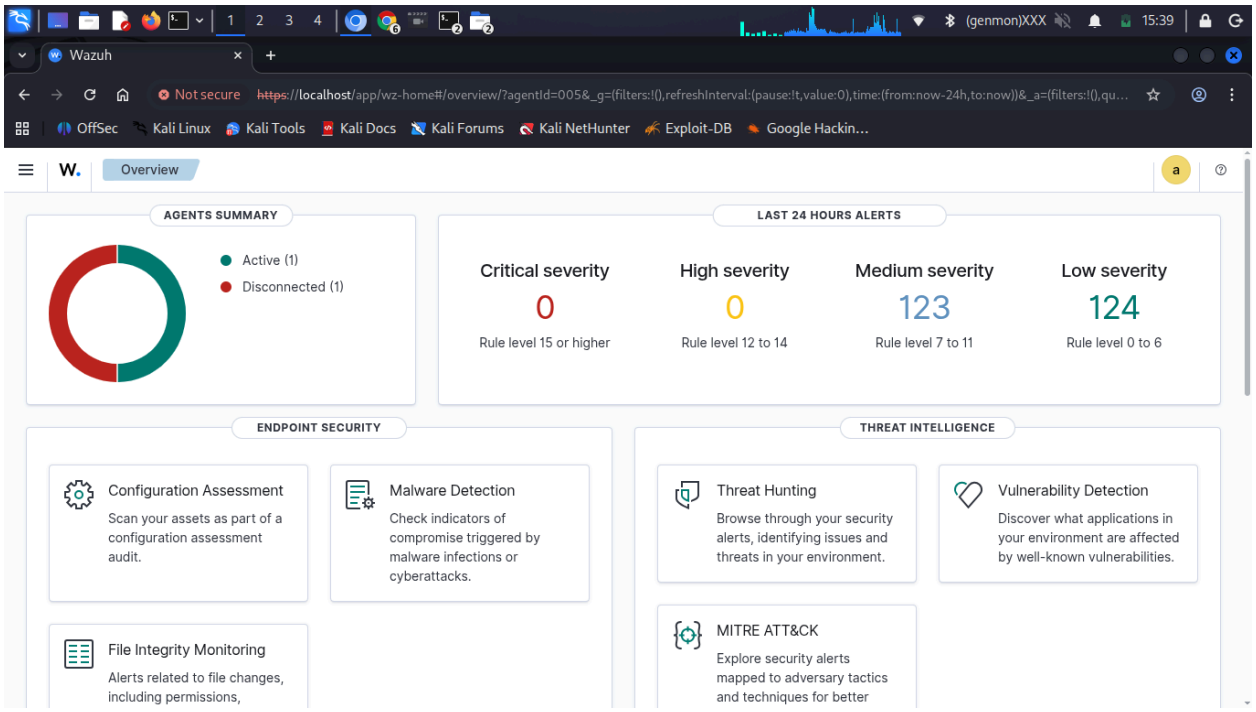
1. Objective

The primary goal of this task was to deploy the Wazuh Agent on a Kali Linux endpoint and establish communication with the Wazuh Manager (hosted in a Docker environment). A key requirement was to configure File Integrity Monitoring (FIM) to detect unauthorized changes to critical system files and specific sensitive directories in real-time, serving as a foundation for advanced threat detection.

2. Implementation Strategy

Step 1: Wazuh Deployment & Connectivity:

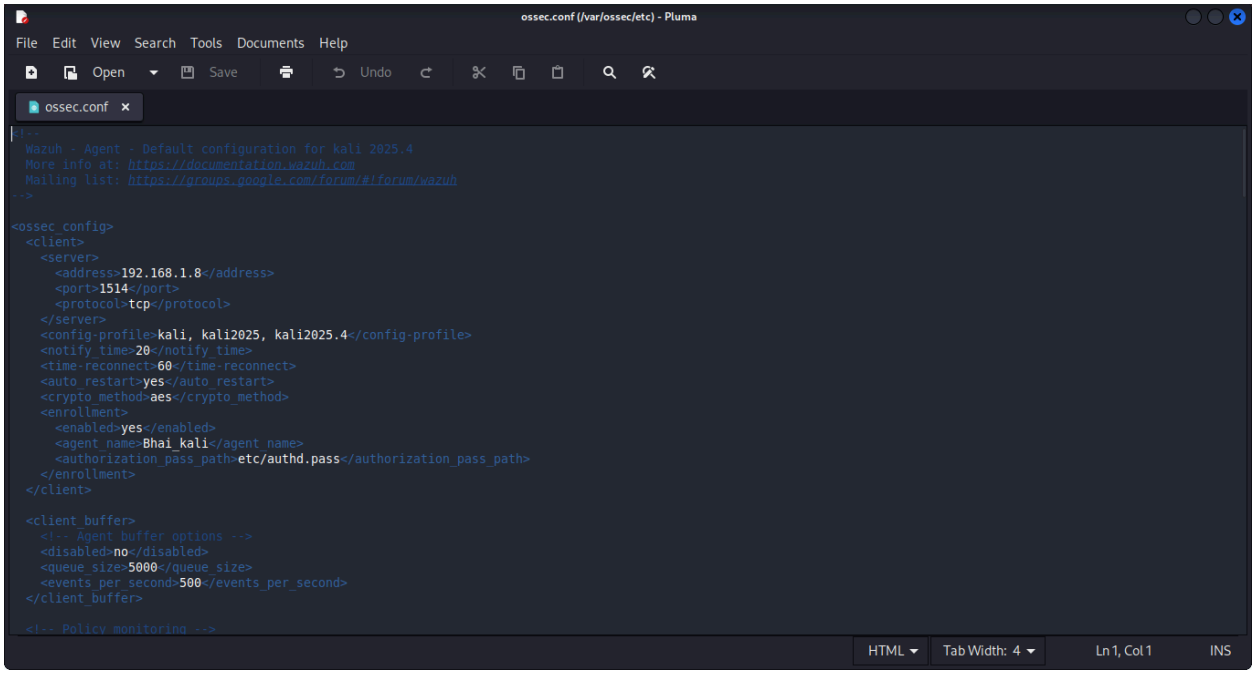
Firstly, I learned basic architecture about Wazuh and its components, then I moved toward deployment of its components by just simply following the Wazuh official manual (I install the Wazuh Server on a Docker). Then I installed the Wazuh Agent on the Kali Linux machine. Initially, I attempted to connect the agent to 127.0.0.1 (localhost), but this resulted in connection failures because the Wazuh Manager was running inside a Docker container.



```
┌──(bhai㉿bhai)-[~]
└─$ sudo docker compose ls

NAME                STATUS              CONFIG FILES
single-node         running(3)          /home/bhai/Desktop/Wazuh/wazuh-docker/single-node/docker-compose.yml

┌──(bhai㉿bhai)-[~]
└─$ cd   /home/bhai/Desktop/Wazuh/wazuh-docker/single-node/

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker/single-node]
└─$ ls
config  docker-compose.yml  generate-indexer-certs.yml  README.md

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker/single-node]
└─$ sudo docker-compose ps
NAME                          IMAGE                          COMMAND                  SERVICE           CREATED      STATUS          PORTS
single-node-wazuh.dashboard-1   wazuh/wazuh-dashboard:4.14.1   "/entrypoint.sh"         wazuh.dashboard   5 days ago   Up 7 hours      443/tcp, 0.0.0.0:443->5601/tcp,
[::]:443->5601/tcp
single-node-wazuh.indexer-1     wazuh/wazuh-indexer:4.14.1     "/entrypoint.sh open…"   wazuh.indexer     5 days ago   Up 7 hours      0.0.0.0:9200->9200/tcp, :::9200-
>9200/tcp
single-node-wazuh.manager-1     wazuh/wazuh-manager:4.14.1     "/init"                  wazuh.manager     5 days ago   Up 24 minutes   0.0.0.0:1514-1515->1514-1515/tcp
, :::1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, :::514->514/udp, 0.0.0.0:55000->55000/tcp, :::55000->55000/tcp, 1516/tcp

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker/single-node]
└─$
```

```
bhai@bhai: ~/Desktop/Wazuh/wazuh-docker/single-node
Session  Actions  Edit  View  Help

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker]
└─$ ls
build-docker-images  CHANGELOG.md  docs  indexer-certs-creator  LICENSE  multi-node  README.md  SECURITY.md  single-node  tools  VERSION.json  wazuh-agent

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker]
└─$ cd wazuh-agent

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker/wazuh-agent]
└─$ ls
config  docker-compose.yml

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker/wazuh-agent]
└─$ cd ..

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker]
└─$ cd multi-node

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker/multi-node]
└─$ ls
config  docker-compose.yml  generate-indexer-certs.yml  Migration-to-Wazuh-4.4.md  README.md  volume-migrator.sh

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker/multi-node]
└─$ cd single-node
cd: no such file or directory: single-node

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker/multi-node]
└─$ cd ..

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker]
└─$ cd single-node

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker/single-node]
└─$ ls
config  docker-compose.yml  generate-indexer-certs.yml  README.md

┌──(bhai㉿bhai)-[~/Desktop/Wazuh/wazuh-docker/single-node]
└─$
```

- **Resolution:** To resolve the network isolation issue, I configured the agent to communicate via the host's LAN IP address (192.168.1.8) on port 1514, which successfully established the connection.

- 

Step 2: Configuring File Integrity Monitoring (Syscheck):

I modified the agent's configuration file (/var/ossec/etc/ossec.conf) to enable the **Syscheck** module. I defined specific policies to monitor the integrity of critical system binaries and configuration files.

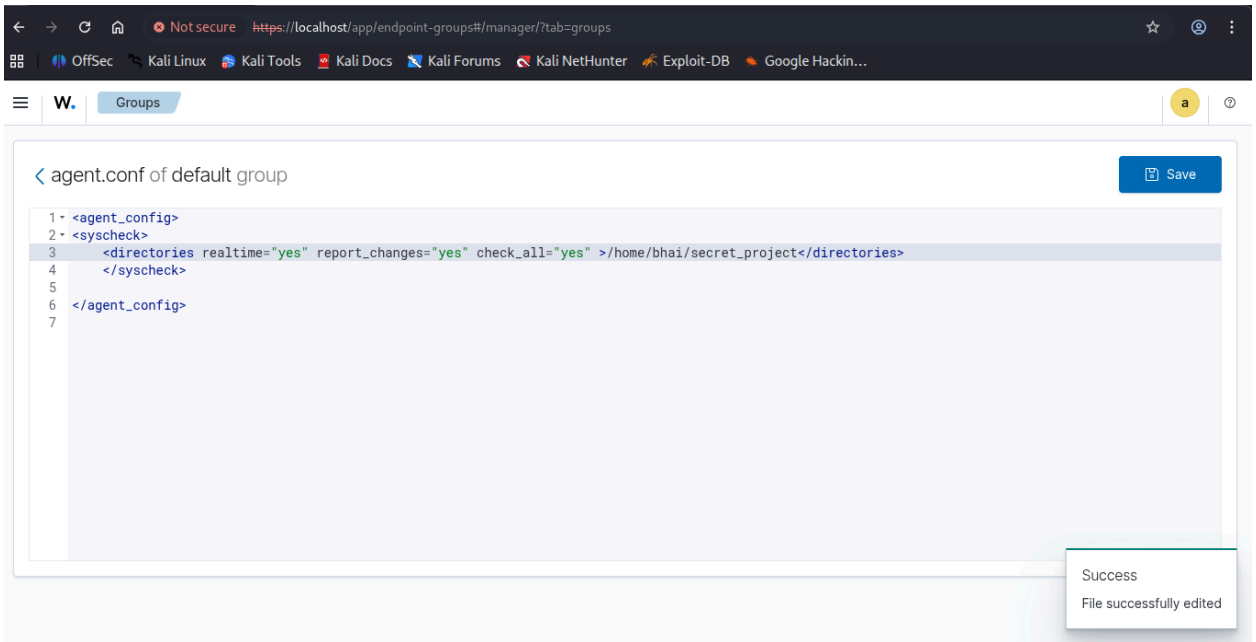
My configuration focused on three security layers:

1. **Critical System Files:** Monitored /etc, /usr/bin, and /sbin to detect if an attacker tries to modify system configurations or replace binaries with backdoors.
2. **Kernel/Boot Files:** Monitored /boot to protect the startup process.
3. **Target Directory:** Specifically monitored /home/bhai/secret_project as per the task requirements.(I made this Directory just to check my configured settings)

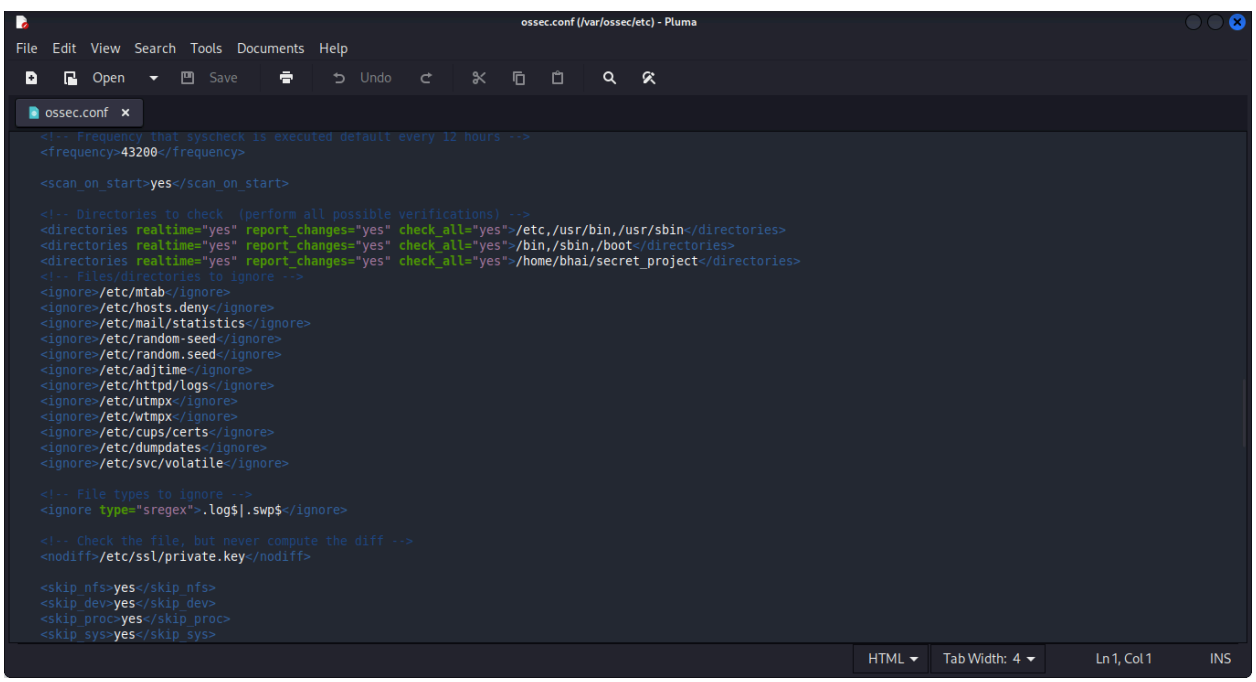
Key Configuration Parameters Used: I utilized the following XML attributes to ensure maximum visibility:

- realtime="yes": To ensure alerts are triggered the exact second a file is modified, rather than waiting for a scheduled scan.
- report_changes="yes": To capture the exact content added or removed from a file (providing forensic evidence).
- check_all="yes": To verify file attributes such as permissions, ownership, and size, ensuring that even if content isn't changed, permission tampering is detected.

FIM Rule update from WAZUH SERVER ENDPOINT



FIM Rule update from WAZUH CLIENT ENDPOINT



3. Tuning & Reducing False Positives

To prevent "alert fatigue" and reduce noise in the logs, I implemented specific exclusion rules. Without these, the system would generate alerts for harmless, routine operations.

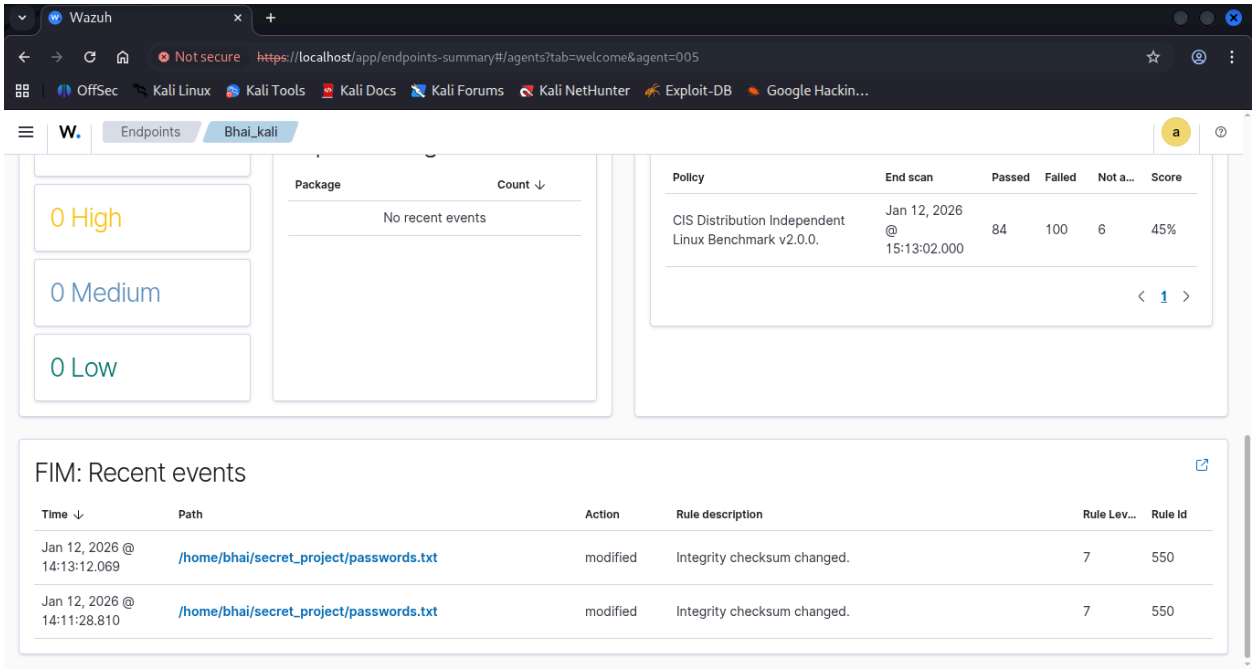
- **Ignored Extensions:** I used regex (type="sregex") to ignore .log and .swp files, as these change constantly during normal system use.
- **Volatile Files:** Explicitly ignored /etc/mtab and /etc/adjtime since these files are updated automatically by the OS and do not indicate a security threat.
- **Sensitive Data Protection:** Used the <nodiff> tag for SSL private keys to alert on changes without storing the sensitive key content in the dashboard logs.

To be Honest I did nothing in this section because all the ignores were already in place and could not find anything more to be ignored.

4. Testing & Validation

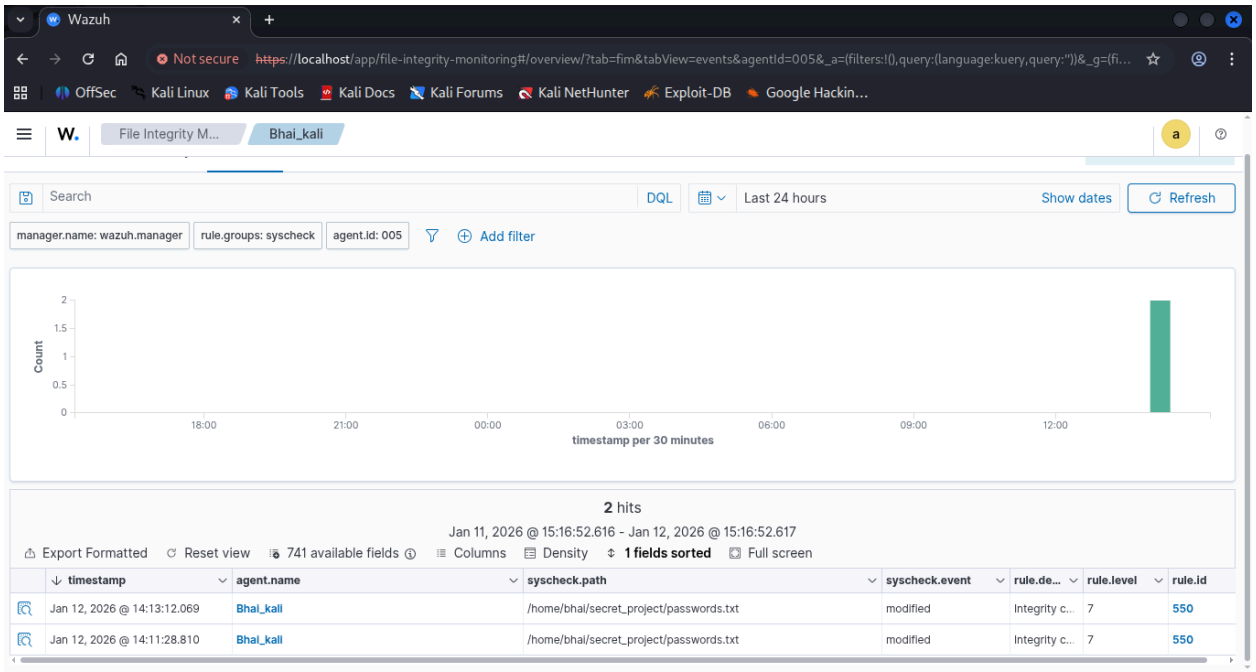
To verify the system was working, I performed a controlled test:

1. I created a sensitive file named passwords.txt inside the monitored directory.
2. I executed a command to append text to the file: echo "Internship Task Complete" >> passwords.txt.
3. **Result:** The Wazuh Dashboard immediately triggered a **Level 7 Alert (Rule ID: 550 - Integrity checksum changed)**. The alert details confirmed the file path, the user who made the change, and the exact content modification.



5. Challenges Faced

- **Docker Networking:** The initial connection failed because the Docker container loopback address was not accessible from the host agent. This was resolved by switching to the Host IP.
- **XML Syntax Errors:** During configuration, I encountered a service failure due to spacing issues in the XML tags (e.g., realtime ="yes" vs realtime="yes"). I learned that ossec.conf is strict about syntax, and resolved this by cleaning the code and validating the tags before restarting the service.

6. Conclusion

The Wazuh Agent is now fully operational with active real-time monitoring. The FIM module is successfully distinguishing between authorized system noise and potential security threats, fulfilling the security monitoring requirements of the task.